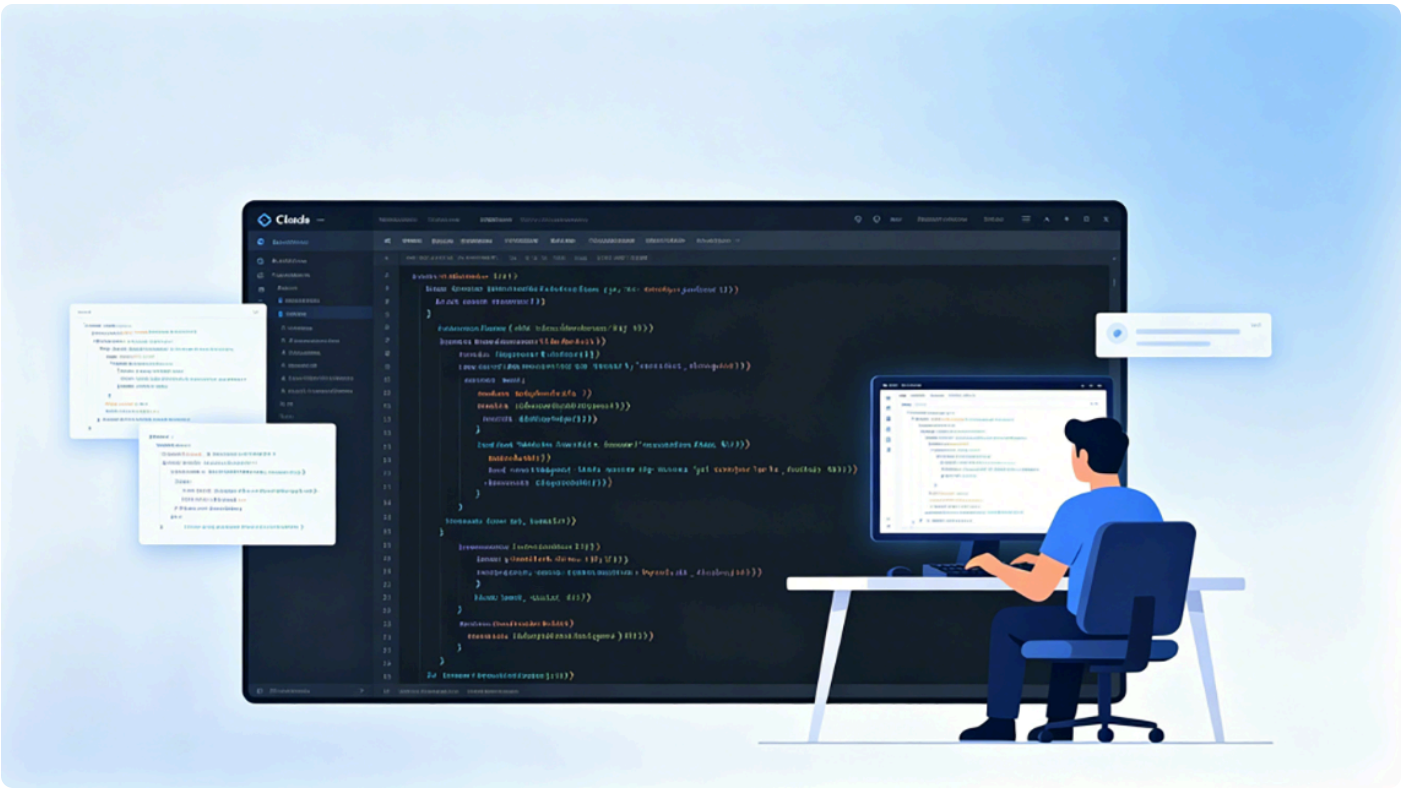
06 | Claude Code 实战必备：CLAUDE.md、Skills 与工程规范

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

前三讲我们做了产品定义、架构设计，过程中一直在往 CLAUDE.md 里写东西——03 讲写了项目概述，04 讲写了应用架构和代码组织，05 讲写了部署架构和数据库规范，你已经在用 Claude Code 了，但我们还没有系统性地聊过它本身的机制和技巧。这一讲补上这块。**你每天都在用的工具，多了解一层，效率会提高很多。**

可能会有同学讲，CLAUDE.md 不是很简单吗？不是有 Skill 吗？这些事是不是放到 Skill 讲效果更好。在我看来，不管是 CLAUDE.md，还是 Skill，都只是通过不同的方式将要遵守的规范告诉大模型。

所以，这节课我们会从最基础的 CLAUDE.md 开始讲起，里面放一些基础的规范。然后在后续的课程中融入 Skill，让你体验到各个维度工具的优势。

CLAUDE.md：你和 AI 的合作协议

它是什么？**一个放在项目根目录下的 Markdown 文件**。Claude Code 每次启动新会话时自动读取，你不需要手动喂。它就是你和 Claude Code 之间的“合作协议”——项目是什么、代码怎么组织、有哪些规矩。

它的生效范围？　项目根目录的 CLAUDE.md 对整个项目生效。你也可以在子目录下放 CLAUDE.md，它只对那个目录下的代码生效，比如 hify-web/ 下放一份前端专属的规范，hify-chat/ 下放一份对话引擎的特殊约定。子目录的规范会和根目录的叠加，不会覆盖。

它的结构怎么组织？　前三讲我们已经实践了从宏观到微观的五层：

1. 项目上下文：这是什么项目，做什么不做什么（第 03 讲）
2. 架构规范：模块划分、依赖关系、外部调用处理（第 04 讲）
3. 代码组织规范：分层结构、Controller/Service/Mapper 职责（第 04 讲）
4. 部署与数据库规范：部署架构、索引规则、分页规范、pgvector（第 05 讲）
5. 接口规范与行为指令：这一讲补完

前四层已经写好了，第五层还缺，现在就补上。

因为下面两部分，很通用，基本拿来即用。所以我直接给出内容。你也可以尝试和 Claude Code 一起生成下面两部分的内容。

接口规范

复制代码

```
## 接口规范

### 路径
RESTful 风格：/api/v1/{资源复数名}
GET    /api/v1/providers           # 列表（分页）
POST   /api/v1/providers           # 创建
GET    /api/v1/providers/{id}      # 详情
PUT    /api/v1/providers/{id}      # 更新
DELETE /api/v1/providers/{id}      # 删除
POST   /api/v1/providers/{id}/test-connection  # 非 CRUD 操作用动词

```

```
### 统一响应
所有接口返回 Result<T>：
{ "code": 200, "message": "success", "data": {...} }

### 分页
请求：page（从 1 开始）、pageSize（默认 20，最大 100）
响应：Result<PageResult<T>>，PageResult 包含 list、total、page、pageSize

### 空值
- 列表字段空时返回 []，不返回 null
- 字符串字段空时返回 ""，不返回 null
- 对象不存在时返回 null

### 错误码
四位数字，按模块分段：
1000-1999 通用 | 2000-2999 Provider | 3000-3999 Agent
4000-4999 Chat | 5000-5999 MCP | 6000-6999 Workflow | 7000-7999 Knowledge
```

行为指令

行为指令是 CLAUDE.md 里最容易被忽略但最有用的部分。它不是约束代码风格，是约束 Claude Code 的“行为模式”——遇到不同情况该怎么行事。

复制代码

```
## 行为指令

### 写代码时
- 每个功能用最简单直接的方式实现
- 不引入不必要的设计模式，除非我明确要求
- 不做过度抽象
- 不引入技术栈以外的依赖，需要时先问我
- 所有外部调用必须有超时设置
- 配置项外化到 application.yml，不硬编码

### 改代码时
- 先理解相关模块的设计意图
- 不要为了新功能破坏已有接口契约
- 改完确保已有测试通过

### 不确定时
- 架构选择给我 2-3 个方案对比，我来拍板
- 规范没覆盖的情况，先问我，不要自己编规矩
```

“改代码时”这三条是从实际踩坑中来的。第 02 讲提过，Claude Code 改一个模块时会顺手改别的模块，破坏已有接口。写了这三条之后，它在修改代码前会先说“我理解这个模块的设计意图是……我打算改动……不会影响……”，给你确认的机会。

“不确定时”也很重要。不写这条，Claude Code 遇到规范没覆盖的情况会自己编一套规矩，每次编的还不一样。写了之后，它会主动问你，而不是自作主张。

到这里，CLAUDE.md 的五层全部补完了。

接口契约：模块级的蓝图

CLAUDE.md 里的接口规范定的是通用规矩（路径格式、响应结构、错误码分段）。但每个模块开始开发前，还需要一份具体蓝图——这个模块有哪些接口、每个接口的入参出参是什么。这就是接口契约。

用 Provider 模块举个例子：

复制代码

```
# Provider 模块接口契约

GET /api/v1/providers
  描述：分页查询提供商列表
  参数：page, pageSize, name(可选，模糊搜索)
  响应：Result<PageResult<ProviderListResp>>

POST /api/v1/providers
  描述：创建提供商
  请求体：ProviderCreateReq { name, type, apiKey, baseUrl }
  响应：Result<Long>

DELETE /api/v1/providers/{id}
  描述：删除提供商（需检查是否有 Agent 在使用）
  响应：Result<Void>

POST /api/v1/providers/{id}/test-connection
  描述：测试连通性
  响应：Result<ConnectionTestResp> { success, latencyMs, errorCode, errorMessage }
```

你不需要在项目开始时就把所有模块的接口契约都写完。每个模块开发前写那个模块的就行，这就是第 02 讲说的模块规范。给 Claude Code 这份蓝图，它生成的 Controller 和 DTO 会精确匹配你的设计，不需要大面积返工。

在这一步我希望你学到的，不是 Hify 的接口怎么定义。而是比如你现在已有一个项目，你要添加功能，你怎么快速让 Claude Code 帮你提出当前的规范，并沉淀为 CLAUDE.md，从而让接下来的开发顺起来。或者你全新开发一个项目，怎么快速有你的规范。

Skills：可复用的规范片段

CLAUDE.md 是项目级的全局规范，覆盖整个项目。但有些规范是针对某一类具体任务的，比如怎么写单元测试，怎么做接口契约，怎么做代码 review。这些写进 CLAUDE.md 太细了，不写每次又要重复描述。Skills 解决这个问题。

Skills 是什么？**一个放在 .claude/skills/ 目录下的 Markdown 文件，描述一类任务的标准做法。Claude Code 在执行任务时会自动匹配相关的 Skill**。

什么时候该创建一个 Skill？当你发现自己给 Claude Code 的指令有重复模式时。比如每次写新模块的接口契约，你都要说“RESTful 风格、Result 返回、分页用 PageResult……”。这些就可以沉淀成一个 Skill。

举个例子。后面每个模块开发前都要写接口契约，我把这个过程标准化成一个 Skill：

```
# 接口契约设计 Skill

## 触发条件
当需要为一个新模块设计接口契约时使用。

## 步骤
1. 列出这个模块的核心业务操作（创建、查询、更新、删除、特殊操作）
2. 每个操作定义：HTTP 方法 + 路径 + 入参 + 出参
3. 路径遵循 /api/v1/{资源复数名} 格式
4. 所有接口返回 Result<T>
5. 列表接口支持分页（page，pageSize）
6. 删除接口标注是否需要关联检查
7. 非 CRUD 操作用动词路径（如 /test-connection）
```

创建了这个 Skill 之后，下次你说“帮我设计 Agent 模块的接口契约”，Claude Code 会自动按这个标准流程走，不需要你每次重复描述格式要求。

一期不需要创建太多 Skills。随着开发推进，你会自然发现哪些指令在重复，那就是该创建 Skill 的信号。第 32 讲会讲 Skills 的高阶用法，包括把整个模块交付流程固化成一个 Skill。

用业界规范喂 Claude Code

前面几讲 CLAUDE.md 里的规范都是我们自己定的，但很多规范不需要从零发明，业界已经有成熟的标准，直接用就行。

技巧是：**把业界规范喂给 Claude Code，让它基于这些规范生成适合你项目的版本**。

比如 Java 编码规范。阿里巴巴 Java 开发手册、Google Java Style Guide 都是业界公认的标准。你不需要自己读完几十页再手写，让 Claude Code 消化后帮你提炼：

> 我在做一个 Spring Boot 项目，请基于阿里巴巴 Java 开发手册，帮我提炼出最关键的 20 条编码规范，写成 CLAUDE.md 可以直接用的格式。重点覆盖命名、异常处理、日志、并发这几个方面。不要照搬原文，要精简到 AI 能直接执行。

Claude Code 会生成一份精简版——不是把整本手册塞进去（那太长了，稀释重点），而是提炼出对 AI 写代码最关键的条目。你 review 一遍，删掉不适用的、调整措辞，就能直接用。

同样的思路适用于数据库规范：

> 基于业界 MySQL 设计规范（阿里巴巴 MySQL 规范、互联网公司常见的 MySQL 最佳实践），帮我补充 Hify 项目的数据库规范。当前已有的规范是：[贴上 05 讲写的数据库规范]。帮我看看还有哪些重要的规范遗漏了。

它可能会补充：VARCHAR 长度不超过 5000（超了用 TEXT）、单表字段不超过 30 个、禁止 SELECT *、UPDATE/DELETE 必须带 WHERE 条件。这些你可能知道，但没写进规范，Claude Code 不知道就不会遵守。

这个技巧的本质是：**你不需要是每个领域的专家，但你可以让 Claude Code 帮你把专家的经验转化成你项目的规范。**Java 规范、MySQL 规范、安全规范、API 设计规范，任何一个领域都可以用这个方法。喂入业界标准 ＋ 你的项目上下文，让 Claude Code 生成适配版本，你 review 定稿。

当然，现在业界发展出了很多开发框架，会内置这些规范，但是我希望你知道最底层是什么，而不是直接用别人封装好的。

背后的思维方式

前面都是具体技巧，这一节聊一个更本质的问题：**怎么判断一条规范该不该写进 CLAUDE.md**？

格式不难，难的是你要知道该写什么。

1. **从架构决策推导规范**。第 04 讲选了模块化单体，目的是以后能拆分微服务。从这个决策推导出“跨模块走 Service 接口，不直接引用其他模块的 Mapper”。这条规范不是凭空想出来的，是架构决策的必然要求。你写 CLAUDE.md 时，每条规范都应该能追溯到一个架构决策或一个具体问题。追溯不到，就没必要写。

2. **规范颗粒度跟着问题走**。Claude Code 反复跑偏的地方写细，不跑偏的地方不写。“文件编码用 UTF-8”，它基本不会搞错，不值得占篇幅。“Controller 不写业务逻辑”，它经常犯，必须写。你的注意力放在它真正容易犯错的地方。
3. **规范是长出来的，不是设计出来的**。Hify 的 CLAUDE.md 从第 03 讲的半页纸到现在的完整版，没有一条是提前想出来的，全部是在实际开发中遇到问题后补上去的。第 02 讲的 SDD 闭环就是这个过程，每次 AI 跑偏，问自己是不是规范没覆盖到，然后补上。接受规范是渐进式完善的，不要追求一次写完。

总结就是：让业界标准规范为基准，逐步补充满足项目自身特点的规范。

让 Claude Code 帮你生成完整 CLAUDE.md

最后一个实操环节。前面几讲我们逐步填充了 CLAUDE.md 的各个部分，加上这一讲补完的接口规范和行为指令，现在让 Claude Code 把它们合并成一份完整版。

> 我在做一个叫 Hify 的项目，以下是前期做的所有决策：[贴上 03–05 讲写进 CLAUDE.md 的所有片段]。另外请基于阿里巴巴 Java 开发手册，补充编码规范部分。请帮我合并生成一份完整的 CLAUDE.md。要求：结构清晰，从项目概述到行为指令，规范要具体到 AI 能直接执行。

Claude Code 会生成一份完整的初稿。大概能覆盖 80% 的内容，剩下 20% 需要你修改。具体的内容我这里就不放了。实际效果你可以用上面的 Prompt 去 Claude Code 中运行，看下前后的对比。

我拿到初稿后做了几处调整：

> 调整了行为指令的措辞。它写的是：建议使用简单的实现方式。我改成：每个功能用简单直接的方式实现，不引入不必要的设计模式，除非我明确要求。更具体、更强硬。AI 理解“建议”和“必须”的区别。
>
> 删掉了多余的内容。它加了一段详细的 logback 配置。这个不是 AI 容易跑偏的地方，不值得占篇幅，删了。

补上了它遗漏的边界。不做什么，它写得比较保守。我补上了完整的列表：不做多租户、不做计费、不做微调、不做 WebApp 发布。边界越明确，Claude Code 越不会好心办坏事。

总结

这一讲系统性地讲了 Claude Code 的核心机制和实战技巧。

CLAUDE.md 是你和 AI 的合作协议。五层结构从宏观到微观，项目上下文→架构→代码组织→数据库→接口 + 行为指令。根目录放全局规范，子目录放模块规范，两者叠加生效。

Skills 是可复用的规范片段。发现指令在重复，就该创建 Skill。

业界规范喂入，是写 CLAUDE.md 的加速器。不需要自己从零写每条规范，把阿里巴巴 Java 规范、MySQL 最佳实践喂给 Claude Code，让它生成适合你项目的版本。

思维方式比具体技巧更重要。每条规范追溯到一个架构决策，颗粒度跟着问题走，接受规范是渐进完善的。

到这里，顶层设计篇就完成了：产品定义（03）、架构设计上（04）、架构设计下（05）、Claude Code 实战技巧（06）。CLAUDE.md 写完了，Claude Code 对 Hify 的全部认知就位了。从下一讲开始，我们将进入工程搭建——初始化项目工程，让 Claude Code 搭建完整的脚手架。

思考题

试着用“业界规范喂入”的技巧，找一份你熟悉的业界规范（阿里巴巴 Java 手册、Google Style Guide 或者你公司内部的编码规范），让 Claude Code 基于它生成一份适合你当前项目的 CLAUDE.md 编码规范部分。对比它生成的版本和你自己写的版本，它补充了哪些你没想到的？它生成的哪些条目其实不适用于你的项目需要删掉？

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (8)

最新　精选

Geek_f882d9
2026-04-09 来自加拿大

能否借鉴《Claude Code 工程化实战》的https://github.com/huangjia2019/claude-code-engingeering，把每一讲的中间过程（与claude的交互记录、最终生成的文件），分别归档。

作者回复: 这个后面会有一个完整的放在github上，能运行的仓库分享给大家。

6



$侯
2026-04-09 来自浙江

假设我的项目有上百个接口，要全部写进接口契约中吗

作者回复: 不需要，因为接口的格式只会有一个。就跟以前你会跟前端约定的规范/格式是一样的。现在你+前端+AI约定，然后AI写出代码，前端就能用。然后前端也用同一份规范让AI写前端代码。就闭环了

2



梦想的优惠券
2026-04-18 来自湖北

同一份md规范文档，在不同模型上运行，结果可能不一样，比如某些业务差异详细原因的分析，A模型可以分析到具体数据的根源；换了B模型，可能就输出一个模糊的原因。这种解决方式是只能去调整我的md文件吗？还是说是因为模型的原因，比如B模型偷懒，没有完全读取我的md文件说明

作者回复: 在我看来是模型的原因，我们不得不承认，不同模型的能力是由差异的。而且差距蛮大的。

1



cricket1981
2026-04-12 来自上海

既有项目引入claude code，如何让其阅读既有项目代码生成CLAUDE_CODE.md文件初稿，我再在此基础上修改确认

作者回复: 这个我一般会把我手上已有的所有资料扔给cc（原因主要是省事，我比较懒，能分门别类，资料齐全更好）。
然后让它深度理解代码，生成一份。
然后我再调整，调整过程挺重要的，因为cc很多细节会理解错，或者瞎猜。
这一步能做好，后面的事情做起来其实挺快的。就跟带一个聪明的啥都不会的技术大佬一样
调整过程就是沉淀类似docs/spec/claude.md的过程。

共 2 条评论>

1

Geek_347b1f
2026-04-10 来自贵州

请问老师这个专栏hify会最终实现到几期，很期待可以跟着老师学做成工业级的

作者回复: 如果大家感兴趣，需要的话，可以一直实现的。我的目的是让大家能够学到东西。我也觉得实现这个挺好玩的。

1



J
2026-05-02 来自福建

kills 是可复用的规范片段。发现指令在重复，就该创建 Skill。　为什么不是直接写在CLAUDE.md规范

作者回复: 遇到改写skill的时候就会写。只要是重复的需求就应该写Skill。这个不需要显示写在CLAUDE.md中

共 3 条评论>

tta
2026-04-08 来自山西

催更, hh



菲尔德

2026-04-07 来自贵州

顶楼！